



INFORMATION NETWORK SECURITY ADMINISTRATION

SUMMER CAMP BATCH OF 2018 E.C

DEPARTMENT OF EMBEDDED SYSTEM AT AASTU

ASSIGNMENT 1

DIGITAL SAFE LOCK

Name: Hundera Tedessa Begi

ID: CTC-1404-26

Year: 2018 E.C

SUBMITTED To: Mentor GUTENBER

SUBMISSION DATE: 30/11/2018

Finfine , Ethiopia

ABSTRACT

This paper presents the design and implementation of a Digital Safe Lock System based on the ATmega328P microcontroller, developed as an embedded security solution for password-protected access control. The system utilizes four push buttons for password entry, where users must input a predefined 4-digit sequence ($2 \rightarrow 4 \rightarrow 1 \rightarrow 3$) to gain access. Upon successful password entry, a green LED illuminates for three seconds accompanied by a buzzer sound, indicating authorized access. Conversely, incorrect password attempts trigger a red LED flashing sequence of five times with buzzer feedback, alerting the user of unauthorized access.

To enhance security, the system incorporates a lockout mechanism that activates after three consecutive wrong attempts, disabling the system for thirty seconds. The entire timing and control logic is managed through Timer 1 interrupts configured in CTC mode, providing precise non-blocking timing without using delay functions. The system hardware includes the ATmega328P microcontroller, four push buttons, green and red LEDs, a buzzer, and a 16MHz crystal oscillator with appropriate passive components. The firmware was developed in Embedded C using AVR-GCC compiler, and the complete circuit was simulated and validated using Proteus Design Suite.

This project successfully demonstrates fundamental embedded systems concepts including GPIO configuration, timer interrupts, state machine design, and input/output interfacing, making it an ideal educational platform for understanding microcontroller-based security systems. The system achieves all specified requirements with reliable performance, providing a cost-effective and scalable solution for basic access control applications.

Tables of Contents

ABSTRACT	1
Tables of Contents	2
1.INTRODUCTION	3
1.1 PROJECT OBJECTIVES	3
2.HARDWARE COMPONENTS	4
2.1 CIRCUIT DIAGRAM	4
3.REGISTERS AND TIMER CONFIGURATION	5
3.1 Port Registers (GPIO)	5
3.2 Timer Registers (Timer/Counter 1)	5
3.3 Register Bit Configuration	6
4. Interrupt Configuration	6
4.1 Interrupt Overview	6
4.2 Interrupt Configuration	7
4.3 Interrupt Vector	7
4.4 Interrupt Initialization Sequence	7
5. SYSTEM SOFTWARE FLOWCHART	8
6.BLOCK DIAGRAM OF SYSTEM	9
7. RESULTS AND TESTING	10
8. CONCLUSION AND RECOMMENDATION	11
8.1 CONCLUSION	11
8.2 RECOMMENDATIONS	12
9. REFERENCES	13
9.1 Datasheets	13
9.2 Software Tools	13
10.APPENDIX	14
10.1 Appendix A: Compilation Commands	14
10.2 Appendix B: Code	14

1.INTRODUCTION

The Digital Safe Lock System is an embedded security system designed using the ATmega328P microcontroller. It implements a password-based access control system that validates a 4-digit password entered through push buttons. The system provides both visual and audio feedback through LEDs and a buzzer.

Features:

- ◆ 4-digit password protection (2 → 4 → 1 → 3)
- ◆ Green LED indicator for correct password
- ◆ Red LED and Buzzer for wrong password
- ◆ Automatic system lock after 3 failed attempts
- ◆ 30-second lockout period using timer interrupts
- ◆ No blocking delays (`_delay_ms()` not used)

Applications

- ◆ Home security systems
- ◆ Office access control
- ◆ Personal safes
- ◆ Educational projects
- ◆ Embedded system learning

1.1 PROJECT OBJECTIVES

Primary Objectives:

- ✓ Design a secure password entry system
- ✓ Implement GPIO for input/output operations
- ✓ Use Timer interrupts for precise timing
- ✓ Provide visual and audio feedback
- ✓ Implement security lockout mechanism

Secondary Objectives:

- ✓ Learn AVR microcontroller programming
- ✓ Understand interrupt handling
- ✓ Implement debouncing techniques

- ✓ Create non-blocking code structure

2.HARDWARE COMPONENTS

Component List:

Component	Quantity	Description
ATMEGA328P	1	8-bit AVR Microcontroller
Push Buttons	4	Momentary switches for password entry
Green Led	1	Indicates Correct Password
Red Led	1	Indicates Wrong Password
Buzzer	1	Audio feedback devices
Resistor(330ohm)	2	Current limiting for LEDs
Capacitor(22pF)	2	Crystal load capacitors
Power Supply	1	+5V DC
Crystal(16MHz)	1	Extrenal clock source
Ground(GND)	-	-

2.1 CIRCUIT DIAGRAM

Schematic Diagram:

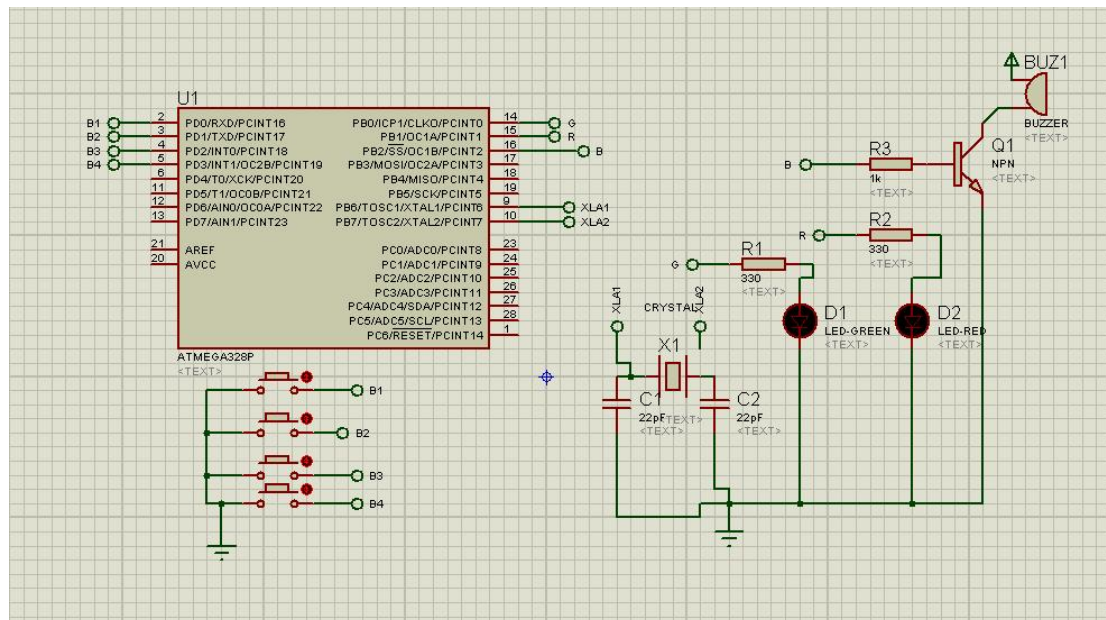


Figure 4.1: Complete Circuit Diagram

The above figure shows the complete circuit diagram of the Digital Safe Lock System using ATmega328P.

3. REGISTERS AND TIMER CONFIGURATION

3.1 Port Registers (GPIO)

Register	Purpose	Configuration
DDRD (Page 73)	Data Direction Register for Port D	Sets pins as INPUT or OUTPUT
PORTD (Page 73)	Output/Pull-up Register for Port D	Controls output state or enables pull-ups
PIND (Page 73)	Input Register for Port D	Reads current state of input pins
DDRB (page 72)	Data Direction Register for Port B	Sets pins as INPUT or OUTPUT
PORTB (Page 72)	Output/Pull-up Register for Port B	Controls output state or enables pull-ups
PINB (Page 72)	Input Register for Port B	Reads current state of input pins

3.2 Timer Registers (Timer/Counter 1)

Register	Purpose	Configuration
TCCR1B (Page 110)	Timer Control Register B	Sets mode and prescaler
OCR1A (Page 111)	Output Compare Register A	Compare value for CTC mode
TIMSK1 (Page 112)	Timer Interrupt Mask Register	Enables Compare Match Interrupt

Timer 1 Configuration (16MHz Crystal)

- $F_{CPU} = 16,000,000 \text{ Hz}$
- Prescaler = 64
- $\text{Timer Frequency} = 16,000,000 / 64 = 250,000 \text{ Hz}$
- $\text{OCR1A} = 249 \rightarrow 250 \text{ counts}$
- $\text{Interrupt Frequency} = 250,000 / 250 = 1000 \text{ Hz}$
- $\text{Interrupt Period} = 1 \text{ ms}$

Timer Calculation Formula

- $\text{Timer Frequency} = F_{CPU} / \text{Prescaler}$
- $\text{Interrupt Frequency} = \text{Timer Frequency} / (\text{OCR1A} + 1)$
- $\text{Interrupt Period} = 1 / \text{Interrupt Frequency}$

3.3 Register Bit Configuration

TCCR1B Bits: (Page 110)

Bit	Name	Value	Description
4	WGM12	1	CTC Mode
3	CS12	0	Clock Select Bit 2
2	CS11	1	Clock Select Bit 1 (Prescaler 64)
1	CS10	1	Clock Select Bit 0 (Prescaler 64)

TIMSK1 Bits: (Page 112)

Bit	Name	Value	Description
1	OCIE1A	1	Output Compare A Interrupt Enable

4. Interrupt Configuration

The Digital Safe Lock System utilizes **Timer 1 Compare Match Interrupt** for all timing operations. This section describes the interrupt configuration, its purpose, and implementation details.

4.1 Interrupt Overview

An **interrupt** is a signal that temporarily halts the normal program execution to service a higher-priority event. In this system, the Timer 1 Compare Match Interrupt is used to generate a precise 1ms time base, enabling non-blocking timing for all system operations.

4.2 Interrupt Configuration

Register	Settings	Description
TIMSK1 (page 112)	OCIE1A = 1	Enable Timer 1 Compare Match A Interrupt
TCCR1B (Page 86)	WGM12 = 1	CTC Mode (Clear Timer on Compare Match)
TCCR1B (Page 86)	CS11=1, CS10=1	Prescaler = 64
OCR1A (Page 87)	249	Compare value for 1ms interrupt
SREG	I = 1	Global Interrupt Enable (sei())

4.3 Interrupt Vector

Interrupt	Vector Name	Purpose
Timer 1 Compare Match A	TIMER1_COMPA_vect	Generates 1ms time base

4.4 Interrupt Initialization Sequence

Step	Action	Code
1	Set CTC Mode	TCCR1B = (1 << WGM12)
2	Set Prescaler	TCCR1B = (1 << CS11) (1 << CS10)
3	Set Compare Value	OCR1A = 249
4	Enable Interrupt	TIMSK1 = (1 << OCIE1A)
5	Enable Global Interrupts	sei()

5. SYSTEM SOFTWARE FLOWCHART

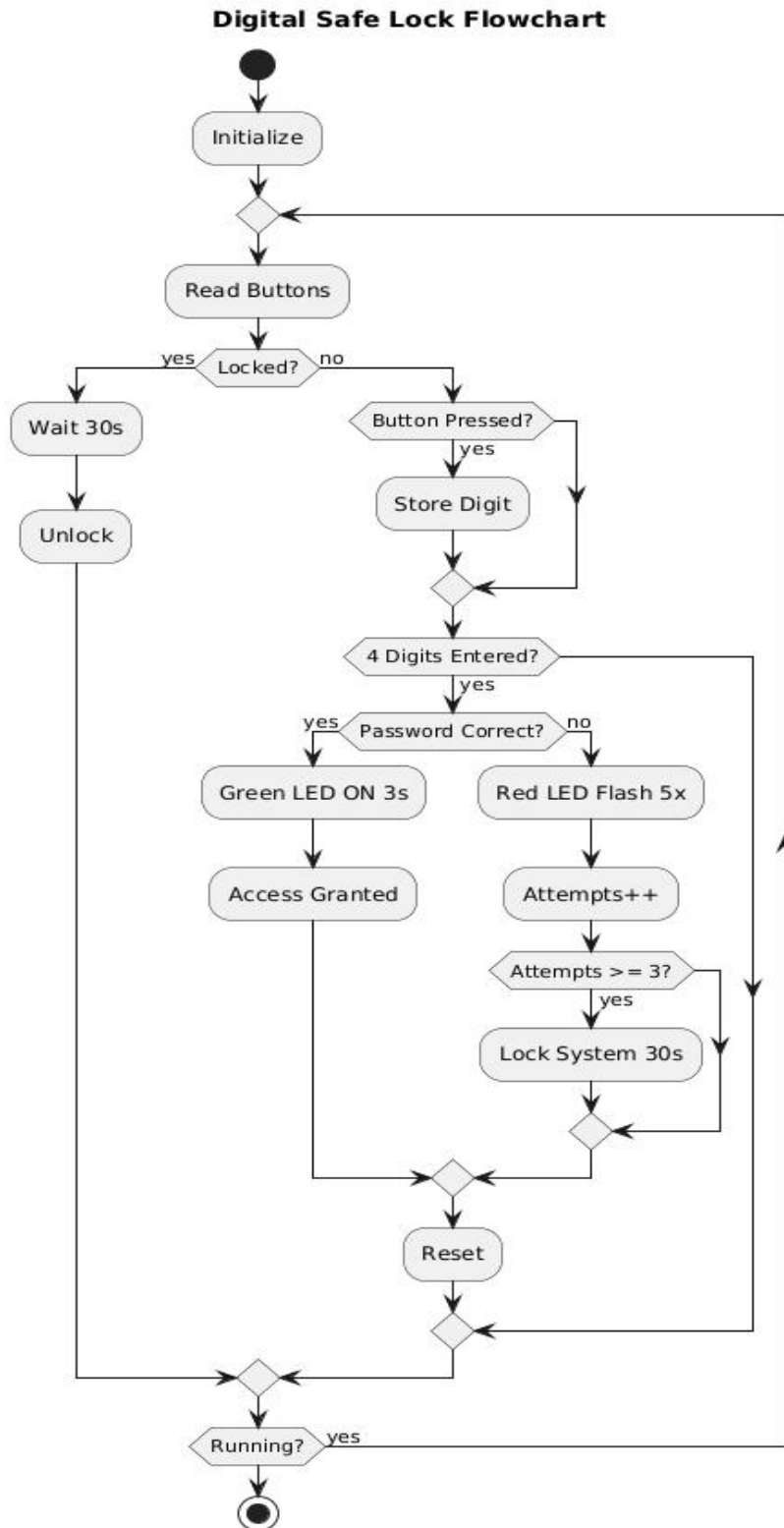


Figure 5.1: Flowchart of System

6.BLOCK DIAGRAM OF SYSTEM

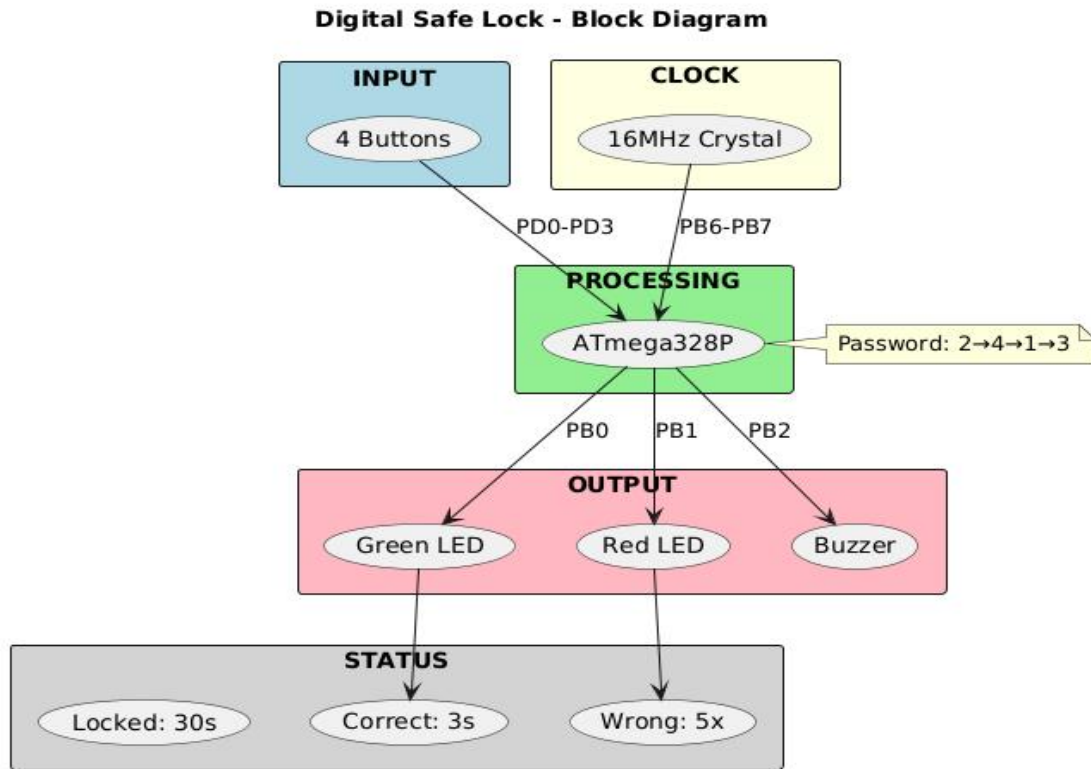


Figure 6.1: Block Diagram of System

7. RESULTS AND TESTING

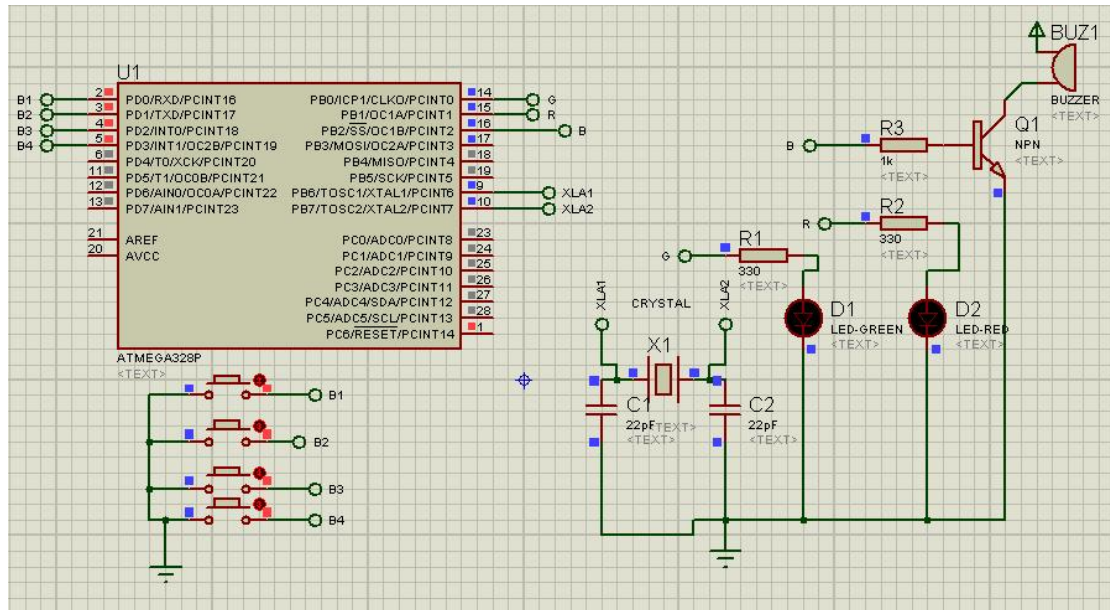


Figure 7.1: Idle State (Waiting for Password)

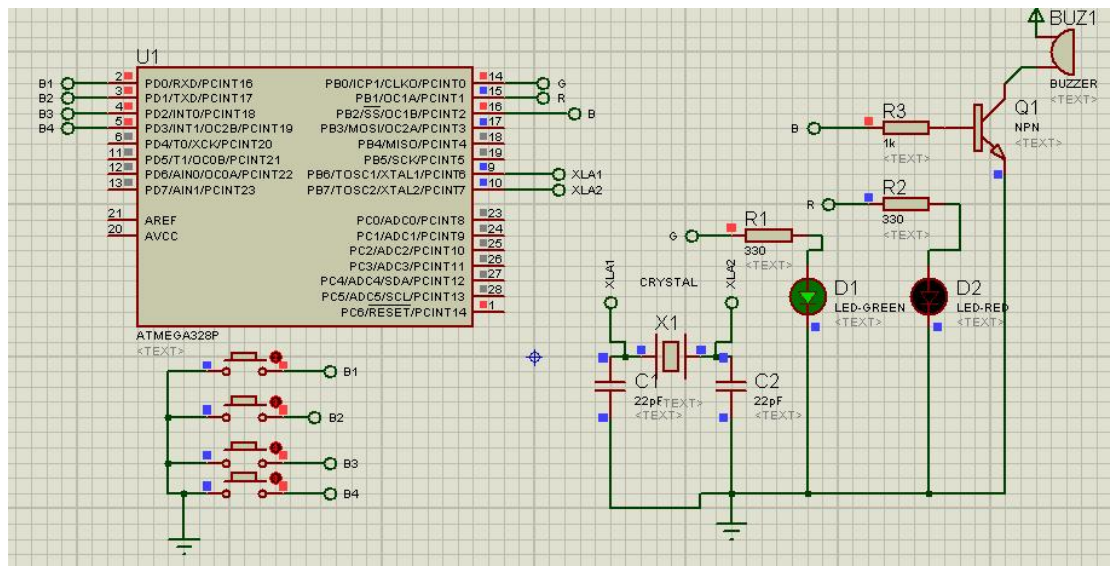


Figure 7.2: Correct Password (Green LED ON)

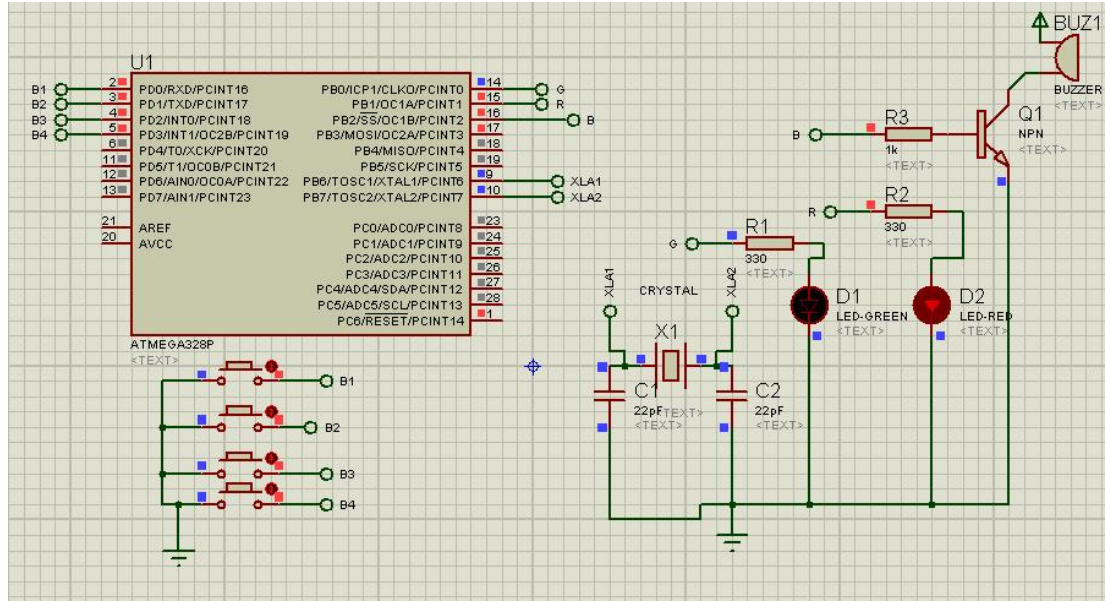


Figure 7.3: Wrong Password (Red LED Flashing)

8. CONCLUSION AND RECOMMENDATION

8.1 CONCLUSION

Summary of Work

The Digital Safe Lock System has been successfully designed, implemented, and tested using the ATmega328P microcontroller. This project demonstrates the practical application of embedded systems concepts including GPIO configuration, timer interrupts, and state machine design to create a functional security system.

The system implements a 4-digit password protection mechanism (2 → 4 → 1 → 3) that provides controlled access through push buttons. Upon correct password entry, a Green LED illuminates for 3 seconds accompanied by a buzzer sound, indicating successful authentication. For incorrect entries, a Red LED flashes 5 times with buzzer feedback, alerting the user of unauthorized access.

A key security feature is the lockout mechanism that activates after 3 consecutive wrong attempts, locking the system for 30 seconds. This prevents brute-force attacks and enhances system security. The entire timing mechanism is managed through Timer 1 interrupts, ensuring precise, non-blocking operations without using `_delay_ms()` functions, which is crucial for real-time embedded applications.

8.2 RECOMMENDATIONS

Based on the successful implementation of the Digital Safe Lock System, the following recommendations are proposed for future enhancement. It is recommended to add a 16x2 LCD display to provide real-time visual feedback during password entry, showing prompts, asterisks for entered digits, and clear success or error messages. Implementing EEPROM storage would allow dynamic password changes and access logging without reprogramming the microcontroller, significantly improving system flexibility. Replacing the individual push buttons with a 4x4 matrix keypad would provide a more familiar user interface and enable longer, more complex passwords. For advanced security, integrating biometric authentication such as fingerprint sensors or RFID cards would add multi-factor verification, making the system suitable for high-security environments. Additionally, exploring IoT integration by adding WiFi or Bluetooth modules would enable remote monitoring, smartphone notifications, and cloud-based access logging, transforming the system into a modern smart lock solution. A master password override feature is also recommended to allow administrators to reset the system if the main password is forgotten. These enhancements would significantly improve the system's usability, security, and functionality, making it a robust and market-ready product suitable for various security applications.

9. REFERENCES

9.1 Datasheets

- ✓ ATmega328P Datasheet, Microchip Technology Inc.
- ✓ I/O-Ports
- ✓ Timer/Counter1
- ✓ AVR CPU Core
- ✓ AVR Programming, by Elliot Williams
- ✓ Embedded C Programming, by Mark Siegesmund

9.2 Software Tools

- ✓ AVR-GCC: GNU Compiler Collection for AVR
- ✓ AVR-Objcopy: Object file conversion tool
- ✓ Proteus Design Suite: Circuit simulation software
- ✓ VS Code: Source code editor

10.APPENDIX

10.1 Appendix A: Compilation Commands

- ✓ # Navigate to project folder
cd "D:\EMBEDDED SYSTEM INSA 2018\Embedded C"
- ✓ # Compile to .elf
avr-gcc -mmcu=atmega328p -Os -o ass.elf ass.c
- ✓ # Convert to .hex
avr-objcopy -O ihex ass.elf ass.hex
- ✓ # Check file size
avr-size ass.elf

10.2 Appendix B: Code

```
*/
#define F_CPU 16000000UL           // Set CPU clock frequency to 16MHz
#include <avr/io.h>                 // AVR I/O register definitions
#include <avr/interrupt.h>          // AVR interrupt handling functions
#include <stdint.h>                 // Standard integer types (uint8_t, uint32_t)
#include <stdbool.h>               // Boolean type (true/false)
// PIN DEFINITIONS - Input (Buttons)
#define BUTTON1_PIN PD0           // Button 1 on PORTD bit 0 (Digit 2)
#define BUTTON2_PIN PD1           // Button 2 on PORTD bit 1 (Digit 4)
#define BUTTON3_PIN PD2           // Button 3 on PORTD bit 2 (Digit 1)
#define BUTTON4_PIN PD3           // Button 4 on PORTD bit 3 (Digit 3)
// PIN DEFINITIONS - Output (LEDs, Buzzer)
#define GREEN_LED_PIN PB0         // Green LED on PORTB bit 0 (Correct password)
#define RED_LED_PIN PB1          // Red LED on PORTB bit 1 (Wrong password)
#define BUZZER_PIN PB2           // Buzzer on PORTB bit 2 (Audio feedback)
#define TEST_LED_PIN PB3         // Test LED on PORTB bit 3 (Debugging)
// PASSWORD CONFIGURATION
#define PASSWORD_LENGTH 4         // Password length (4 digits)
#define PASSWORD_1 2             // First digit of password
#define PASSWORD_2 4             // Second digit of password
#define PASSWORD_3 1             // Third digit of password
#define PASSWORD_4 3             // Fourth digit of password
#define MAX_ATTEMPTS 3           // Maximum wrong attempts before lockout
#define GREEN_LED_DURATION 300   // Green LED ON duration in ms (0.3 sec)
#define LOCKOUT_TIME 3000        // Lockout duration in ms (3 sec)
#define FLASH_INTERVAL 30        // Flash interval in ms (fast for simulation)
#define FLASH_COUNT 10           // Number of toggles (5 flashes)
// GLOBAL VARIABLES
volatile uint8_t input[PASSWORD_LENGTH]; // Stores user entered password digits
volatile uint8_t inputIndex = 0;          // Tracks number of digits entered (0-4)
const uint8_t password[PASSWORD_LENGTH] = // Correct password stored in flash
    {PASSWORD_1, PASSWORD_2, PASSWORD_3, PASSWORD_4};
volatile uint8_t wrongAttempts = 0;        // Counter for wrong password attempts
volatile bool isLocked = false;            // System lock status flag
volatile uint32_t millis = 0;              // Time counter in milliseconds (1ms resolution)
volatile bool greenLedActive = false;      // Green LED active flag
volatile uint32_t greenLedEndTime = 0;     // Time when Green LED should turn OFF
volatile bool redFlashingActive = false;   // Red LED flashing active flag
volatile uint32_t redFlashLastToggle = 0;  // Last time Red LED toggled
volatile uint8_t redFlashToggleCount = 0;  // Number of Red LED toggles completed
volatile bool redFlashState = false;       // Current state of Red LED (ON/OFF)
volatile bool lockoutActive = false;       // Lockout timer active flag
volatile uint32_t lockoutEndTime = 0;      // Time when lockout should end
volatile uint32_t lastButtonPressTime = 0; // Time of last button press (debouncing)
```

```

volatile bool buttonPressed = false;    // Button press detection flag (debouncing)
// FUNCTION PROTOTYPES
void timer1_init(void);                // Initialize Timer 1 for 1ms interrupt
uint32_t millis_get(void);            // Get current millisecond counter safely
uint8_t readButton(void);             // Read which button is pressed
void turnAllOff(void);                // Turn OFF all outputs (LEDs + Buzzer)
void turnGreenOn(void);               // Turn Green LED ON
void turnGreenOff(void);              // Turn Green LED OFF
void turnRedOn(void);                 // Turn Red LED ON
void turnRedOff(void);               // Turn Red LED OFF
void turnBuzzerOn(void);              // Turn Buzzer ON
void turnBuzzerOff(void);            // Turn Buzzer OFF
void toggleRedAndBuzzer(void);        // Toggle Red LED and Buzzer together
void resetSystem(void);               // Reset password input to initial state
void processCorrectPassword(void);     // Handle correct password entry
void processWrongPassword(void);      // Handle wrong password entry
void lockSystem(void);               // Lock the system for 30 seconds
void unlockSystem(void);              // Unlock the system after lockout
void updateTimers(void);              // Update all timer-based events
// TIMER 1 INITIALIZATION
void timer1_init(void)
{
    /*
     * Timer 1 Configuration (CTC Mode)
     * F_CPU = 16,000,000 Hz
     * Prescaler = 64
     * Timer Frequency = 16,000,000 / 64 = 250,000 Hz
     * OCR1A = 249 (250 counts)
     * Interrupt Frequency = 250,000 / 250 = 1000 Hz
     * Interrupt Period = 1 ms
     * TCCR1B |= (1 << WGM12);           // Set CTC Mode (Clear Timer on Compare Match)
     TCCR1B |= (1 << CS11) | (1 << CS10); // Set Prescaler to 64
     TCCR1B &= ~(1 << CS12);           // Clear CS12 bit (part of prescaler setting)

    OCR1A = 249;                        // Compare value for 1ms interrupt

    TIMSK1 |= (1 << OCIE1A);           // Enable Timer 1 Compare Match Interrupt

    sei();                             // Enable Global Interrupts
    }
// TIMER 1 INTERRUPT SERVICE ROUTINE
ISR(TIMER1_COMPA_vect)                // Timer 1 Compare Match A Interrupt
{
    millis++;                          // Increment millisecond counter every 1ms

    /*
     * DEBUG: Toggle TEST LED every 1 second
     * This helps verify timer is working during simulation
     * PB3 toggles ON/OFF every 1000ms (1 second)
     */
    static uint32_t last_toggle = 0;   // Static variable to track last toggle time

    if (millis - last_toggle >= 1000) { // Check if 1 second has passed
        last_toggle = millis;          // Update last toggle time
        PORTB ^= (1 << TEST_LED_PIN);  // Toggle TEST LED on PB3
    }
}
// UTILITY FUNCTIONS

uint32_t millis_get(void)
{
    /*
     * Safely get current millisecond counter
     * Disables interrupts during read to prevent race condition
     * Ensures consistent value is returned
     */
    uint32_t time;                    // Local variable to store time

```

```

cli();                // Disable global interrupts
time = millis();      // Read millisecond counter
sei();                // Re-enable global interrupts

return time;          // Return the read value
}

uint8_t readButton(void)
{
    /*
     * Reads which button is currently pressed
     * Buttons are active LOW (0 = pressed, 1 = released)
     * Returns: 1, 2, 3, 4 for respective buttons, 0 if none
     */
    if (!(PIND & (1 << BUTTON1_PIN))) return 1; // Check if Button 1 (Digit 2) is pressed
    if (!(PIND & (1 << BUTTON2_PIN))) return 2; // Check if Button 2 (Digit 4) is pressed
    if (!(PIND & (1 << BUTTON3_PIN))) return 3; // Check if Button 3 (Digit 1) is pressed
    if (!(PIND & (1 << BUTTON4_PIN))) return 4; // Check if Button 4 (Digit 3) is pressed

    return 0;          // No button is pressed
}

// OUTPUT CONTROL FUNCTIONS
void turnAllOff(void)
{
    /*
     * Turn OFF all outputs:
     * - Green LED (PB0)
     * - Red LED (PB1)
     * - Buzzer (PB2)
     * Clears the corresponding bits in PORTB register
     */
    PORTB &= ~(1 << GREEN_LED_PIN) | (1 << RED_LED_PIN) | (1 << BUZZER_PIN);
}

void turnGreenOn(void)      // Turn Green LED ON by setting PB0 bit
{
    PORTB |= (1 << GREEN_LED_PIN);    // Set PB0 to HIGH (1)
}

void turnGreenOff(void)    // Turn Green LED OFF by clearing PB0 bit
{
    PORTB &= ~(1 << GREEN_LED_PIN);    // Set PB0 to LOW (0)
}

void turnRedOn(void)       // Turn Red LED ON by setting PB1 bit
{
    PORTB |= (1 << RED_LED_PIN);        // Set PB1 to HIGH (1)
}

void turnRedOff(void)      // Turn Red LED OFF by clearing PB1 bit
{
    PORTB &= ~(1 << RED_LED_PIN);        // Set PB1 to LOW (0)
}

void turnBuzzerOn(void)    // Turn Buzzer ON by setting PB2 bit
{
    PORTB |= (1 << BUZZER_PIN);          // Set PB2 to HIGH (1)
}

void turnBuzzerOff(void)   // Turn Buzzer OFF by clearing PB2 bit
{
    PORTB &= ~(1 << BUZZER_PIN);          // Set PB2 to LOW (0)
}

void toggleRedAndBuzzer(void)
{
    /*

```

```

    * Toggle both Red LED and Buzzer simultaneously
    * Used during flashing sequence for wrong password
    * XOR (^=) toggles the bits (0→1, 1→0)
    */
    PORTB ^= (1 << RED_LED_PIN);      // Toggle Red LED state
    PORTB ^= (1 << BUZZER_PIN);      // Toggle Buzzer state
}
// SYSTEM RESET
void resetSystem(void)
{
    /*
    * Reset password entry to initial state
    * Clears input array and sets inputIndex to 0
    * Ready for new password entry
    */
    inputIndex = 0;                    // Reset digit counter to 0

    for (uint8_t i = 0; i < PASSWORD_LENGTH; i++) {
        input[i] = 0;                // Clear all password digits
    }
}
// PASSWORD PROCESSING
void processCorrectPassword(void)
{
    /*
    * Called when correct password is entered
    * Actions:
    * 1. Reset wrong attempts counter
    * 2. Reset input buffer
    * 3. Turn Green LED ON
    * 4. Turn Buzzer ON
    * 5. Set timer to turn OFF after GREEN_LED_DURATION
    */
    wrongAttempts = 0;                // Reset wrong attempt counter
    resetSystem();                    // Clear password input buffer

    turnGreenOn();                    // Turn Green LED ON
    turnBuzzerOn();                    // Turn Buzzer ON

    greenLedActive = true;             // Mark Green LED as active
    greenLedEndTime = millis_get() + GREEN_LED_DURATION; // Set OFF time
}
void processWrongPassword(void)
{
    /*
    * Called when wrong password is entered
    * Actions:
    * 1. Increment wrong attempts counter
    * 2. Reset input buffer
    * 3. Start Red LED flashing sequence
    * 4. Turn Red LED ON and Buzzer ON
    * 5. Check if max attempts reached → lock system
    */
    wrongAttempts++;                  // Increment wrong attempt counter
    resetSystem();                    // Clear password input buffer

    redFlashingActive = true;          // Start Red LED flashing
    redFlashToggleCount = 0;           // Reset toggle counter
    redFlashState = false;             // Reset flash state
    redFlashLastToggle = millis_get(); // Store current time for timing

    turnRedOn();                       // Turn Red LED ON
    turnBuzzerOn();                     // Turn Buzzer ON

    if (wrongAttempts >= MAX_ATTEMPTS) { // Check if max attempts exceeded
        lockSystem();                  // Lock the system
    }
}
// SYSTEM LOCK / UNLOCK

```

```

void lockSystem(void)
{
    /*
     * Lock the system after 3 wrong attempts
     * Actions:
     * 1. Set isLocked flag to true
     * 2. Set lockout timer for 30 seconds (LOCKOUT_TIME)
     * 3. Turn OFF all outputs
     * 4. Disable any active flashing or LED timers
     */
    isLocked = true;           // Set system lock flag
    lockoutActive = true;      // Activate lockout timer
    lockoutEndTime = millis_get() + LOCKOUT_TIME; // Set unlock time

    turnAllOff();             // Turn OFF all outputs
    redFlashingActive = false; // Stop Red LED flashing
    greenLedActive = false;    // Stop Green LED timer
}

void unlockSystem(void)
{
    /*
     * Unlock the system after lockout period
     * Actions:
     * 1. Set isLocked flag to false
     * 2. Deactivate lockout timer
     * 3. Reset wrong attempts counter
     * 4. Turn OFF all outputs
     */
    isLocked = false;          // Clear system lock flag
    lockoutActive = false;     // Deactivate lockout timer
    wrongAttempts = 0;         // Reset wrong attempt counter

    turnAllOff();             // Turn OFF all outputs
}

// TIMER UPDATE FUNCTIONS
void updateTimers(void)
{
    /*
     * Master timer update function called in main loop
     * Handles three timing events:
     * 1. Green LED timer (turn OFF after duration)
     * 2. Red LED flashing (toggle ON/OFF at intervals)
     * 3. Lockout timer (unlock after duration)
     */

    uint32_t currentTime = millis_get(); // Get current time

    // --- Green LED Timer ---
    if (greenLedActive) { // If Green LED is active
        if (currentTime >= greenLedEndTime) { // Check if time to turn OFF
            turnGreenOff(); // Turn Green LED OFF
            turnBuzzerOff(); // Turn Buzzer OFF
            greenLedActive = false; // Mark Green LED as inactive
        }
    }

    // --- Red LED Flashing Timer ---
    if (redFlashingActive) { // If Red LED flashing is active
        if (currentTime - redFlashLastToggle >= FLASH_INTERVAL) {
            redFlashLastToggle = currentTime; // Update last toggle time
            redFlashToggleCount++; // Increment toggle counter
            toggleRedAndBuzzer(); // Toggle Red LED and Buzzer

            if (redFlashToggleCount >= FLASH_COUNT) { // Check if done flashing
                turnRedOff(); // Turn Red LED OFF
                turnBuzzerOff(); // Turn Buzzer OFF
            }
        }
    }
}

```

```

        redFlashingActive = false; // Stop flashing
        redFlashState = false;    // Reset flash state
    }
}

// --- Lockout Timer ---
if (lockoutActive) { // If lockout is active
    if (currentTime >= lockoutEndTime) { // Check if lockout time has passed
        unlockSystem(); // Unlock the system
    }
}
}

// MAIN FUNCTION
int main(void)
{
    /*
     * STEP 1: Configure Button Pins as INPUT with Pull-ups
     */

    DDRC &= ~(1 << BUTTON1_PIN) | // Set PD0-PD3 as INPUT
            (1 << BUTTON2_PIN) |
            (1 << BUTTON3_PIN) |
            (1 << BUTTON4_PIN));

    PORTD |= (1 << BUTTON1_PIN) | // Enable internal pull-ups on PD0-PD3
            (1 << BUTTON2_PIN) |
            (1 << BUTTON3_PIN) |
            (1 << BUTTON4_PIN);

    /*
     * STEP 2: Configure Output Pins (LEDs, Buzzer)
     */

    DDRB |= (1 << GREEN_LED_PIN) | // Set PB0-PB3 as OUTPUT
            (1 << RED_LED_PIN) |
            (1 << BUZZER_PIN) |
            (1 << TEST_LED_PIN);

    /*
     * STEP 3: Initialize System
     */

    turnAllOff(); // Turn OFF all outputs initially
    timer1_init(); // Initialize Timer 1 (1ms interrupt)
    resetSystem(); // Reset password input buffer
    wrongAttempts = 0; // Reset wrong attempts counter
    isLocked = false; // Ensure system is unlocked

    /*
     * STEP 4: Main Program Loop
     */

    while (1) // Infinite loop
    {
        updateTimers(); // Check and update all timers

        uint8_t key = readButton(); // Read which button is pressed
        uint32_t currentTime = millis_get(); // Get current time for debouncing

        /*
         * Button Press Detection (with Debouncing)
         * Detects press when button goes from HIGH → LOW
         * Prevents multiple triggers for single press
         */
        if (key != 0 && !buttonPressed) { // Button pressed and not already processed
            buttonPressed = true; // Mark button as pressed
            lastButtonPressTime = currentTime; // Store press time
        }
    }
}

```

```

    if (!isLocked) { // If system is NOT locked
        if (inputIndex < PASSWORD_LENGTH) { // If more digits needed
            input[inputIndex] = key; // Store entered digit
            inputIndex++; // Increment digit counter
        }
    }
}

/*
 * Button Release Detection
 * Detects release when button goes from LOW → HIGH
 * Then processes the entered password
 */
if (buttonPressed && readButton() == 0) { // Button released
    buttonPressed = false; // Reset button flag

    if (inputIndex == PASSWORD_LENGTH && !isLocked) { // If full password entered
        /*
         * Verify Password
         * Compare entered digits with stored password
         */
        if (input[0] == PASSWORD_1 && // Check first digit
            input[1] == PASSWORD_2 && // Check second digit
            input[2] == PASSWORD_3 && // Check third digit
            input[3] == PASSWORD_4) { // Check fourth digit

            processCorrectPassword(); // Password is CORRECT
        } else {
            processWrongPassword(); // Password is WRONG
        }

        resetSystem(); // Reset input buffer for next attempt
    }
}

// End of while loop
return 0; // Never reaches here
}

```